

Keras Framework

A high-level deep learning framework for building and training neural networks

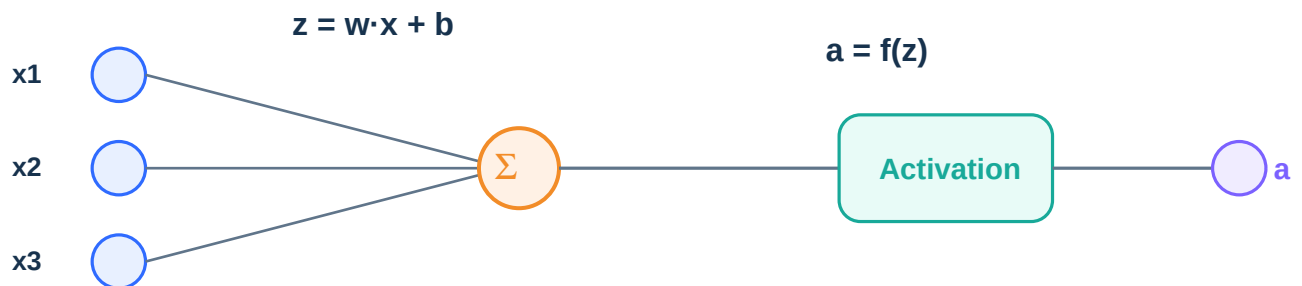
What is Keras?

Keras is a high-level API that makes it easier to create, train, evaluate and deploy neural-network models. It provides reusable building blocks so that developers can focus on the model design instead of writing every low-level tensor operation manually.

Common Keras Building Blocks

Layer	Main purpose
Dense	Fully connected neural-network layer
Conv2D	Extracts visual patterns from images
MaxPooling2D	Reduces spatial size and computation
Flatten	Converts feature maps into a 1D vector
Dropout	Reduces overfitting by randomly disabling units during training
LSTM	Learns patterns in sequences and time-series data

Inside a Neuron



During training: compute loss, gradients and update weights

Core idea: first compute the weighted sum $z = w \cdot x + b$, then apply an activation function $a = f(z)$. During training, Keras calculates the loss, gradients and weight updates automatically.

How a Keras Model Learns

1

Weight initialization

The network starts with small initial weight values.

2

Forward propagation

Input data moves through each layer to produce a prediction.

3

Loss calculation

The prediction is compared with the correct answer.

4

Backpropagation

Gradients show how much each weight contributed to the error.

5

Weight update

An optimizer such as Adam changes the weights to reduce future error.

6

Repeat

The process continues across batches and epochs until performance improves.

Compute options

CPU for small prototypes, **GPU** for faster deep-learning training, and **TPU** for large-scale tensor workloads.

Ready-made Layers

Common choices

Dense - regular neural processing **Conv2D** - images **MaxPooling2D** - reduce feature maps
LSTM - sequences **Embedding** - text **Dropout** - overfitting control **BatchNormalization** - stable training

Three Ways to Build Models

Sequential API

Stack layers one after another

Best for simple linear models

Functional API

Connect layers as a graph

Supports branches, merges and multiple outputs

Model Subclassing

Write your own Python class

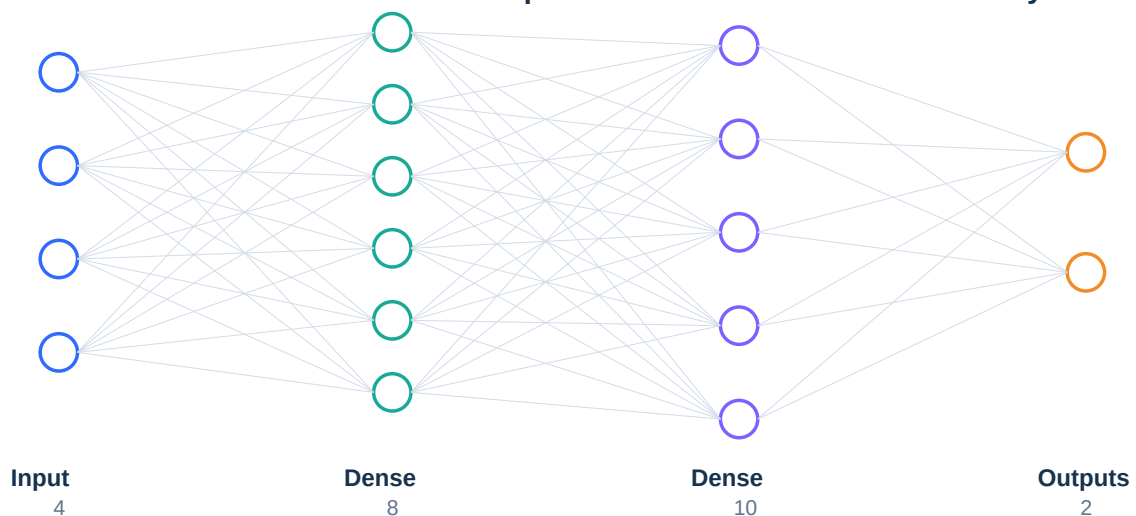
Maximum flexibility and custom behavior

Example: Multi-output Neural Network

The original notes use Iris data and a shared neural network with more than one output.

Input features (X)	Meaning
Sepal length	Numeric feature
Sepal width	Numeric feature
Petal length	Numeric feature
Petal width	Numeric feature

Example: Iris features -> shared hidden layers -> two outputs



Activation	Use
Sigmoid	Binary output: one probability between 0 and 1
Softmax	Multi-class output: probabilities across several classes
Linear	Regression output: unrestricted numeric value

Functional API is useful here

A single input can pass through shared hidden layers and then branch into multiple outputs. For example, one output may perform binary classification using sigmoid while another predicts a continuous value using a linear activation.

Functional API Example

```
from tensorflow.keras import Input, Model
from tensorflow.keras.layers import Dense

inputs = Input(shape=(7,), name='input_layer')
x = Dense(8, activation='relu')(inputs)
x = Dense(10, activation='relu')(x)
x = Dense(10, activation='relu')(x)

output_1 = Dense(1, activation='sigmoid', name='class_output')(x)
output_2 = Dense(1, activation='linear', name='value_output')(x)

model = Model(inputs=inputs, outputs=[output_1, output_2])
```

How to read this model

- `Input(shape=(7,))` means every sample contains seven input features.
- Three Dense layers learn increasingly useful internal representations.
- ReLU is used in hidden layers because it is simple and effective for deep networks.
- The sigmoid output is suitable for a binary decision.
- The linear output is suitable for predicting a numeric value.
- `Model(inputs=..., outputs=...)` creates the final computation graph.

Functional API advantage

It clearly exposes the architecture as a graph. You can build shared layers, branches, skip connections, multiple inputs and multiple outputs without writing a custom class.

Model Subclassing

Subclassing is useful when the architecture or forward pass contains custom Python logic. Instead of only connecting layers, the developer defines a class that inherits from `keras.Model`.

Model Subclassing Example

```
import keras
from keras import layers

class MyModel(keras.Model):
    def __init__(self, num_classes=1, h1=8, h2=10, h3=10):
        super().__init__()
        self.l1 = layers.Dense(h1, activation='relu')
        self.l2 = layers.Dense(h2, activation='relu')
        self.l3 = layers.Dense(h3, activation='relu')
        self.out = layers.Dense(num_classes, activation='sigmoid')

    def call(self, inputs):
        x = self.l1(inputs)
        x = self.l2(x)
        x = self.l3(x)
        return self.out(x)

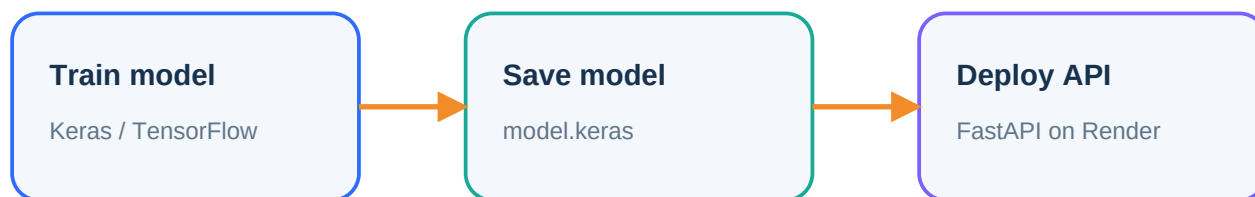
model = MyModel()
```

Approach	Best suited for	Main strength
Sequential API	Simple stack of layers	Fastest and easiest to understand
Functional API	Graphs, branches, multiple inputs/outputs	Flexible and visual model structure
Model Subclassing	Custom behavior and dynamic logic	Maximum control using Python

Important note

In subclassing, layers are usually created in `__init__()`, while the data flow is written inside `call()`. Keras tracks the layer weights automatically.

Saving and Deploying a Keras Model



requirements.txt + runtime.txt + app.py

Save the trained model

```
model.save('iris_mlp.keras')
```

Typical deployment files

File	Purpose
app.py	Loads the model and exposes prediction endpoints
iris_mlp.keras	Saved Keras model
requirements.txt	Python package dependencies
runtime.txt	Optional Python version for the deployment platform

Minimal FastAPI loading example

```
from fastapi import FastAPI
import numpy as np
import keras

app = FastAPI()
model = keras.models.load_model('iris_mlp.keras')

@app.post('/predict')
def predict(features: list[float]):
    x = np.array([features], dtype='float32')
    prediction = model.predict(x)
    return {'prediction': prediction.tolist()}
```

Deployment flow

Train the model locally, save it in the native **.keras** format, place the model beside the API code, install dependencies, and deploy the service to a platform such as Render.